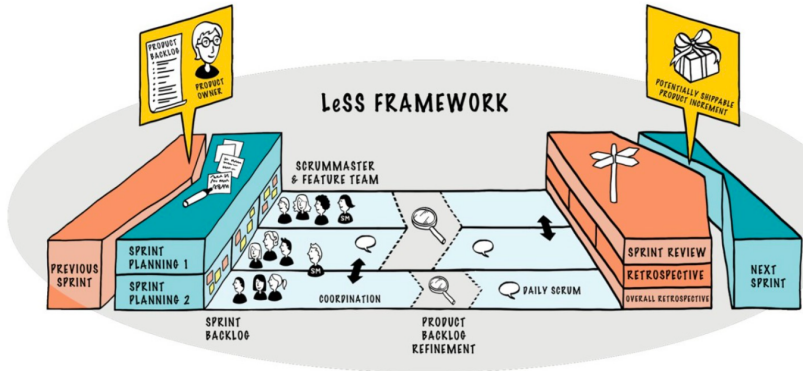
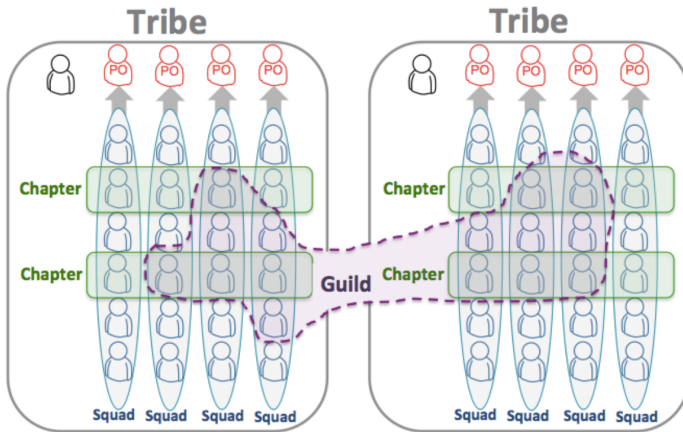


Luento 10

25.11.2025



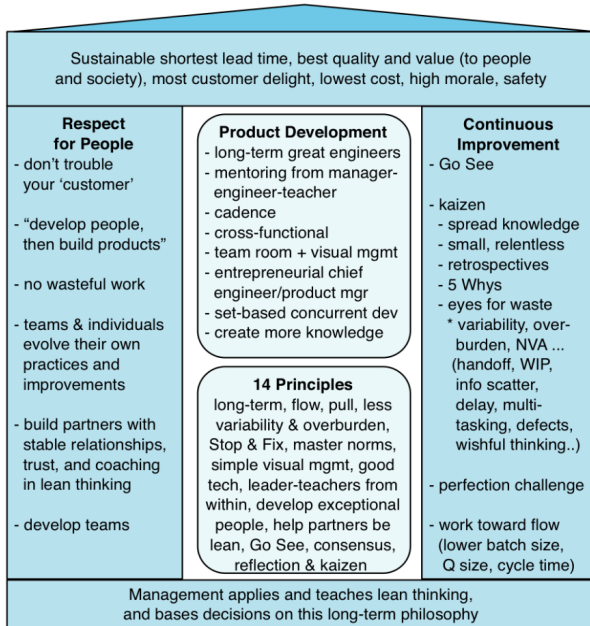
Spotifyn malli



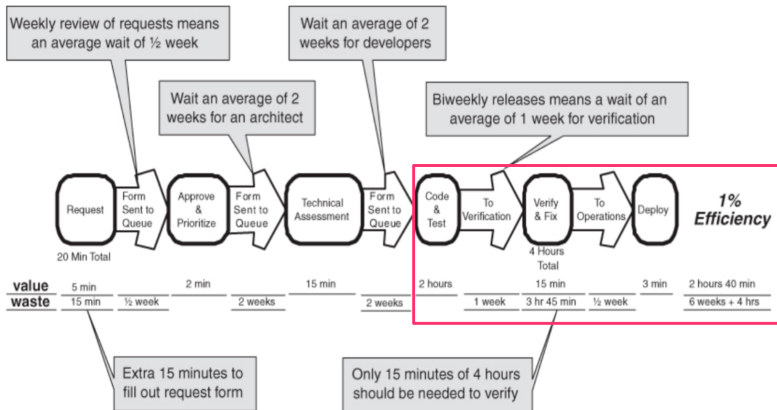
Luento 9

24.11.2025

Lean thinking house



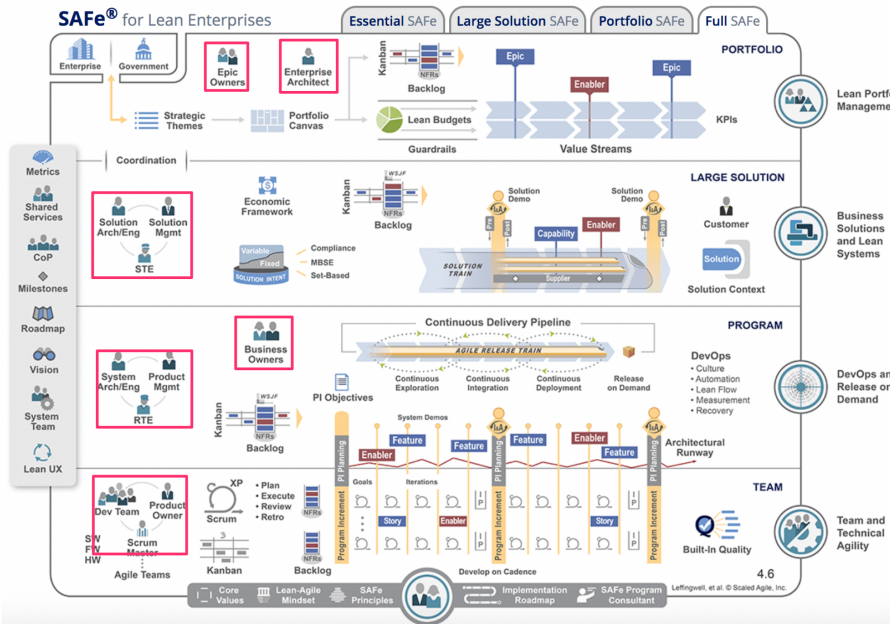
Perimmäisen syyn analyysi: five whys



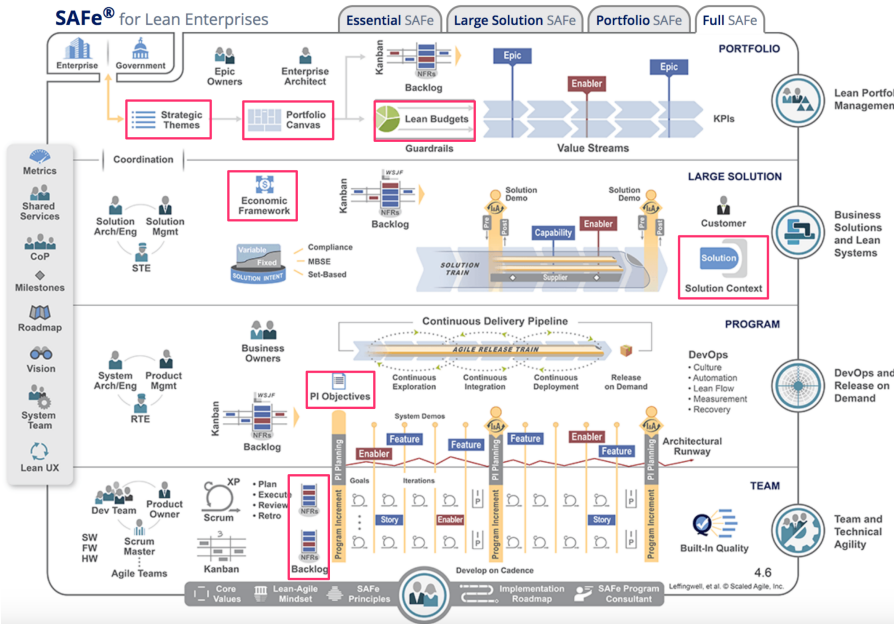
Luento 10

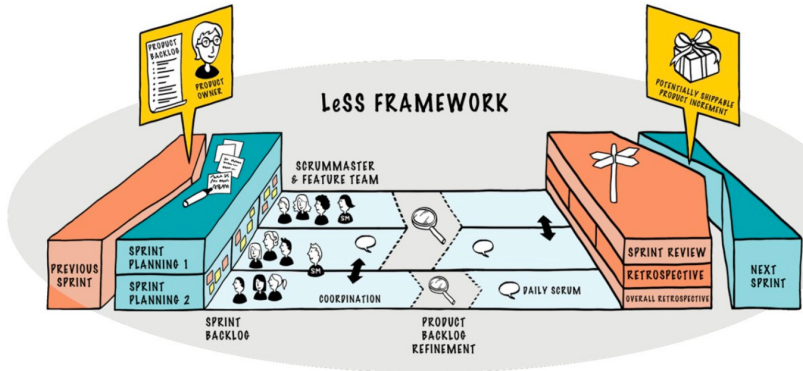
25.11.2025

SAFe: paljon rooleja

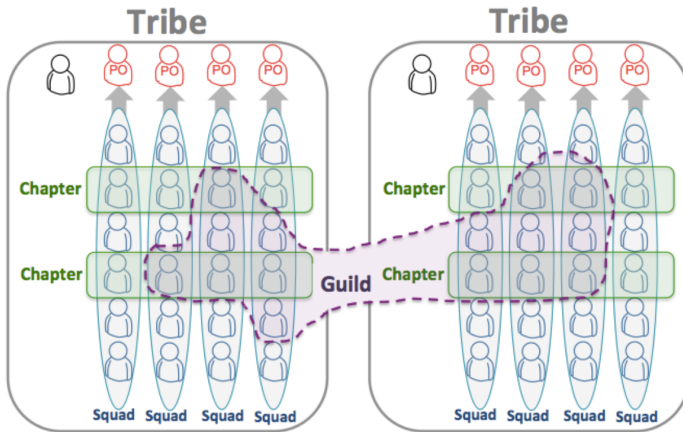


SAFe: paljon artefakteja

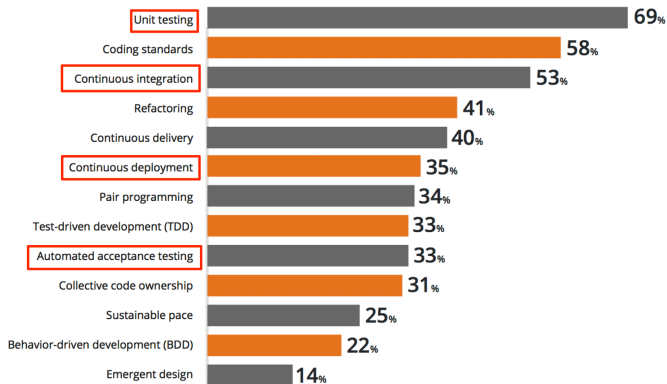




Spotifyn malli



Tekniset käytänteet



Luento 8

18.11.2025

Laadukkaan koodin tuntomerkkejä

- ▶ Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattributteja*, esim. seuraavat:
 - ▶ kapselointi
 - ▶ korkea koheesion aste
 - ▶ riippuvuuksien vähäisyys
 - ▶ toisteettomuus
 - ▶ testattavuus
 - ▶ selkeys

Koheesio Flask-sovelluksessa

```
@app.route("/create_todo", methods=["POST"])
def todo_creation():
    content = request.form.get("content")

    try:
        validate_todo(content)
        create_todo(content)
        return redirect("/")
    except Exception as error:
        flash(str(error))
        return redirect("/new_todo")

# util.py

def validate_todo(content):
    if len(content) < 5:
        raise UserInputError("Todo content length must be greater than 4")

    if len(content) > 100:
        raise UserInputError("Todo content length must be smaller than 100")

# todo_repository.py

def create_todo(content):
    sql = text("INSERT INTO todos (content) VALUES (:content)")
    db.session.execute(sql, { "content": content })
    db.session.commit()
```

Laadukkaan koodin tuntomerkkejä

- ▶ Laadukkaalla koodilla joukko yhteneviä ominaisuuksia, tai *laatuattributteja*, esim. seuraavat:
 - ▶ kapselointi
 - ▶ korkea koheesion aste
 - ▶ riippuvuuksien vähäisyys
 - ▶ toisteettomuus
 - ▶ testattavuus
 - ▶ selkeys

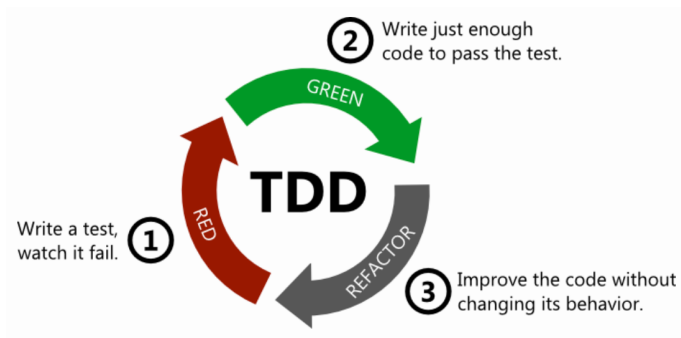
- ▶ teknisen velan määrän kasvu

- ▶ teknisen velan määrän kasvu
- ▶ oppimisvelka

Luento 6

10.11.2025

Test driven development (TDD)



1. Kirjoitetaan sen verran testiä että testi ei mene läpi
2. Kirjoitetaan koodia sen verran, että testi menee läpi
3. Jos huomataan koodin rakenteen menneen huonoksi refaktoroidaan koodin rakenne paremmaksi
4. Jatketaan askeleesta 1

Testit asiakkain kielellä

*** Test Cases ***

Run | Debug | Run in Interactive Console

Login With Correct Credentials

Set Username kalle

Set Password kalle123

Submit Credentials

Login Should Succeed

Run | Debug | Run in Interactive Console

Login With Incorrect Password

Set Username kalle

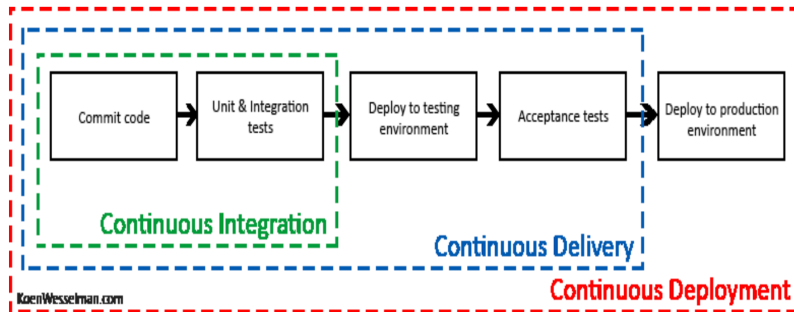
Set Password wrong

Submit Credentials

Login Should Fail With Message Invalid username or password

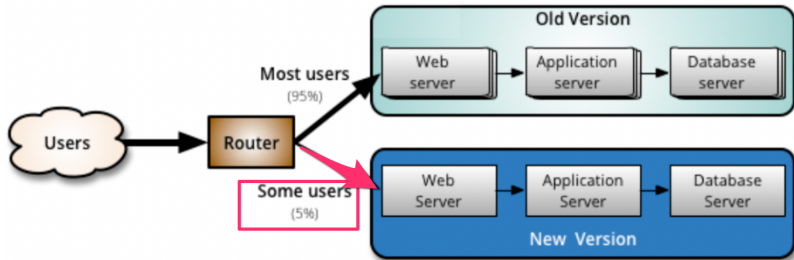
Deployment pipeline

- Vaiheet, joiden suorittaminen edellyttää, että commitattu koodi saadaan siirrettyä staging/tuotantoympäristöön



Canary release

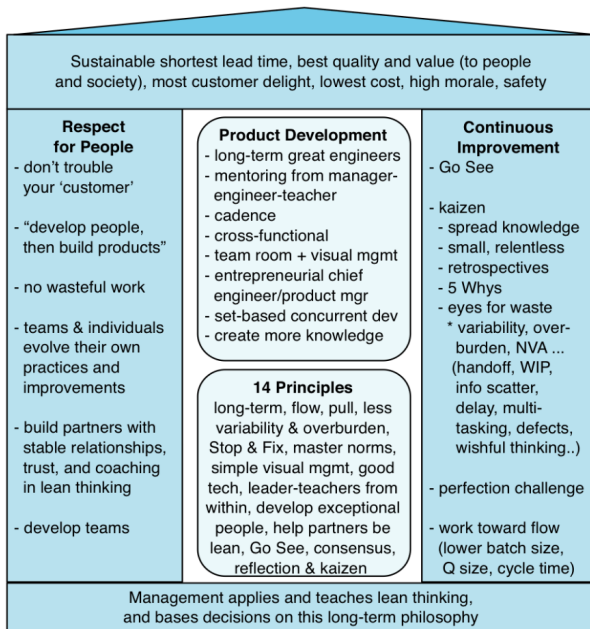
- Kaksi rinnakkaista tuotantoympäristöä, joista uudet ominaisuudet vietään toiseen



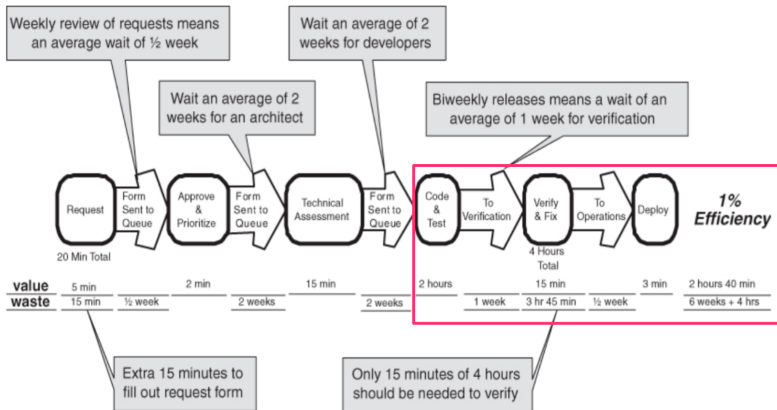
Luento 9

29.11.2023

Lean thinking house



Perimmäisen syyn analyysi: five whys



Luento 1

27.10.2025

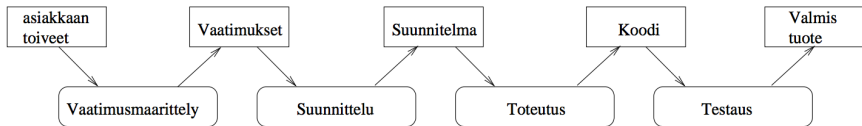
Ohjelmiston elinkaari (software lifecycle)

Riippumatta tyylistä ja tavasta, jolla ohjelmisto tehdään, käy ohjelmisto läpi seuraavat *vaiheet*

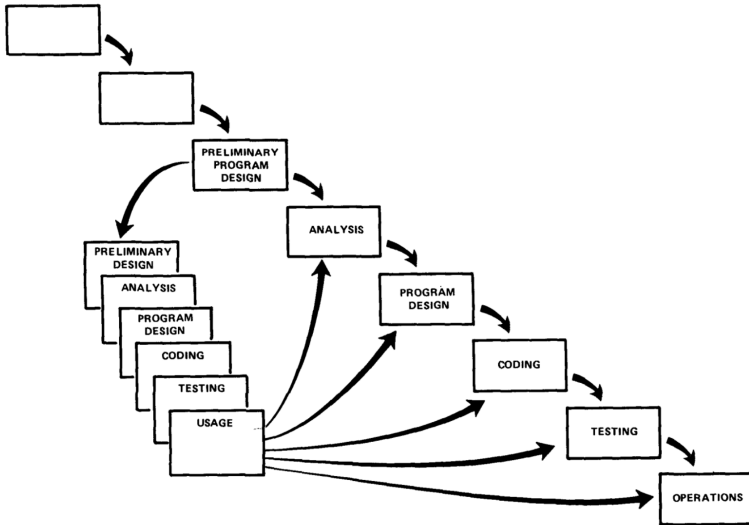
- ▶ Vaatimusten analysointi ja määrittely
- ▶ Suunnittelu
- ▶ Toteutus
- ▶ Testaus
- ▶ Ohjelmiston ylläpito ja evoluutio

Vaiheista muodostuu ohjelmiston “elinkaari”

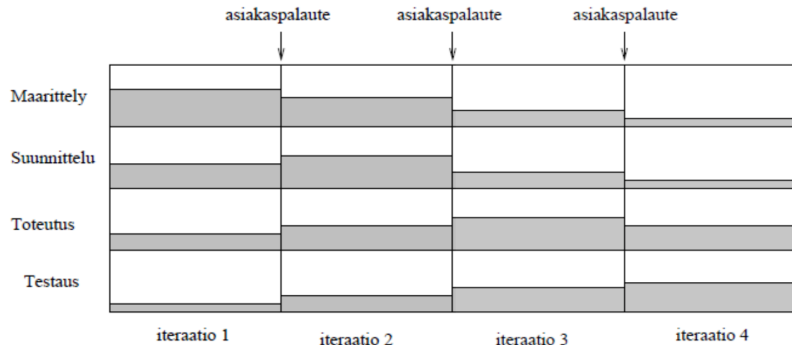
Winston W. Royce: Management of the development of Large Software, 1970



Roycen kahden iteraation malli



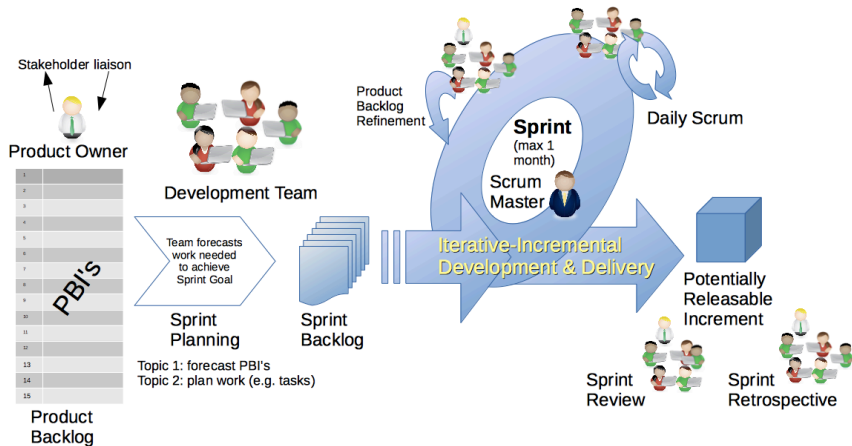
Iteratiivinen ohjelmistokehitys



Luento 2

28.10.2025

Scrum kuvana

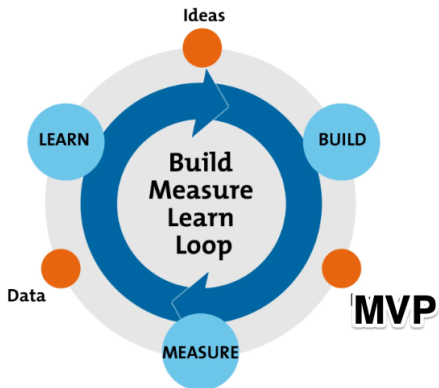


Luento 3

Ohjelmiston elinkaari (software lifecycle)

- ▶ **Vaatimusten analysointi ja määrittely**
- ▶ Suunnittelu
- ▶ Toteutus
- ▶ Testaus
- ▶ Ohjelmiston ylläpito ja evoluutio

Vaatimusmäärittely 2010-luvulla: Lean startup



User story

- ▶ Mike Cohn:
 - ▶ A user story describes ***functionality that will be valuable*** to either user or purchaser of software.
- ▶ User stories are composed of three aspects:
 1. **A written description**
 2. **Conversations**
 3. **Tests**

Hyvän storyn kriteerit

- ▶ Bill Wake *INVEST in good User Stories*, kuusi toivottavaa ominaisuutta
 - ▶ Independent
 - ▶ Negotiable
 - ▶ Valuable to user or customer
 - ▶ Estimable
 - ▶ Small
 - ▶ Testable

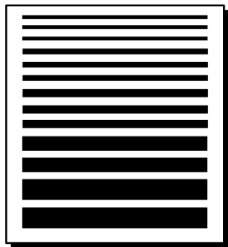
Luento 4

4.11.2025

Hyvä product backlog on DEEP

- ▶ Detailed appropriately
- ▶ Estimated
- ▶ Emergent
- ▶ Prioritized

High
Priority

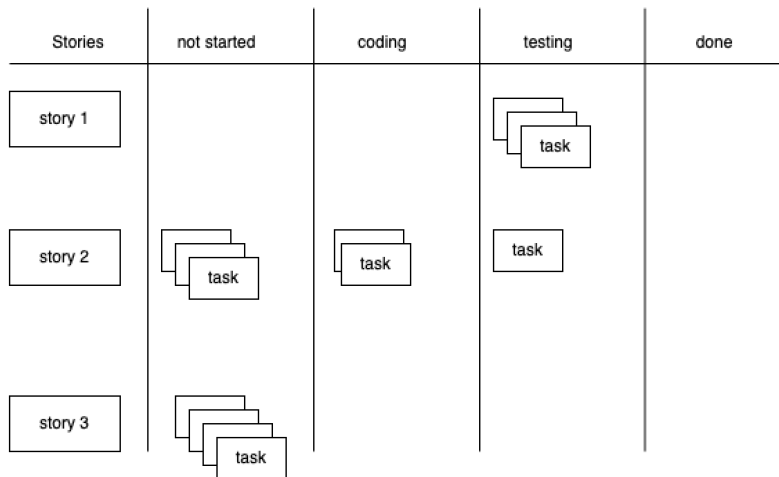


Low
Priority

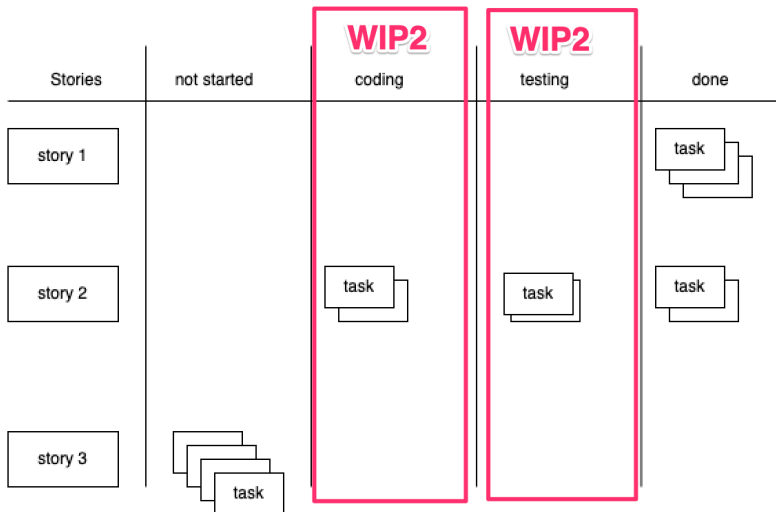
Fine-grained, detailed items ready
to be worked on in the next sprint

Large, coarse-grained items

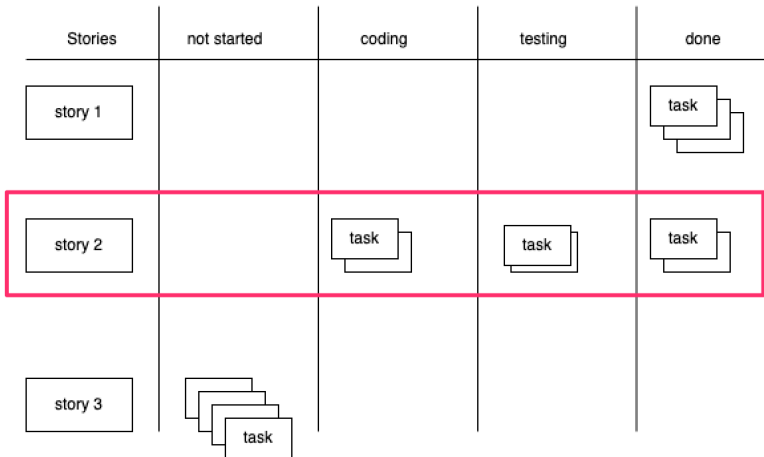
Sprint backlog



WIP-rajoitteen



WIP vain yksi story työn alla



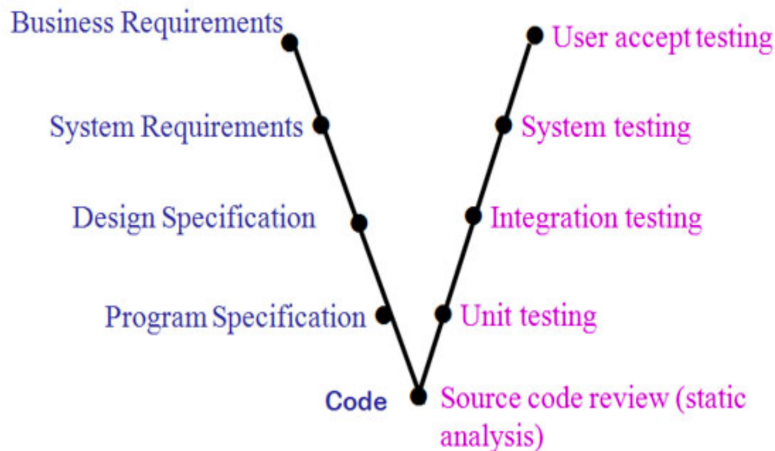
Luento 5

10.11.2025

Testauksen tasot

- ▶ *Yksikkötestaus* (unit testing)
 - ▶ Yksittäisten luokkien, metodien ja moduulien testaus erillään muusta kokonaisuudesta
- ▶ *Integraatiotestaus* (integration testing)
 - ▶ Yksittäin testattujen komponenttien liittäminen yhteen eli integrointi ja kokonaisuuden testaus
- ▶ *Järjestelmätestaus* (system testing)
 - ▶ Toimiiko ohjelmisto vaatimuksiin kirjatulla tavalla?
 - ▶ Tutkii järjestelmää kokonaisuudessaan: *end to end -testaus*
 - ▶ Jakautuu useisiin alalajeihin
- ▶ *Käyttäjän hyväksymistestaus* (user acceptance testing)
 - ▶ Loppukäyttäjän tuotteelle suorittama testaus

“V-malli”



Testisyötteiden valinta: palautussovellus

► Mitä testitapauksia kannattaisi valita palautussovelluksen testaamiseen?

Create a submission for part2

Mark all exercises you have done (check the box if the exercise is done)

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6 ☐ 7 ☐ 8 ☐ 9 ☐ 10 ☐ 11 ☐ 12

Mark all

Clear all

Used hours (reading the material and completing exercises)

GitHub repository

https://github.com/mluukkai/put_your_repository_name_here

Comments

Pressing send will submit this whole part. Any exercises you have not marked done above for this part can **not** be marked done later. If you by accident submit the wrong number of exercises contact the course teacher or Discord admins.

Send

Cancel

Ohtuvarasto: tyhjä, puolitäysi, täysi

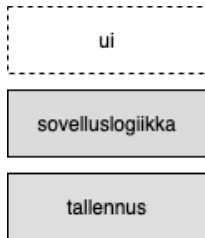
```
class Varasto
    def __init__(self, tilavuus, alku_saldo = 0):
        self.tilavuus = tilavuus
        self.saldo = alkusalto

    def ota_varastosta(self, maara):
        if maara < 0:
            return 0.0

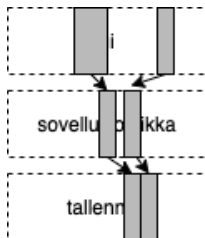
        if maara > self.saldo:
            kaikki_mita_voidaan = self.saldo
            self.saldo = 0.0
            return kaikki_mita_voidaan

        self.saldo = self.saldo - maara
        return maara
```

► Rakenteeseen perustuva integraatio



► Ominaisuuksiin perustuva integraatio

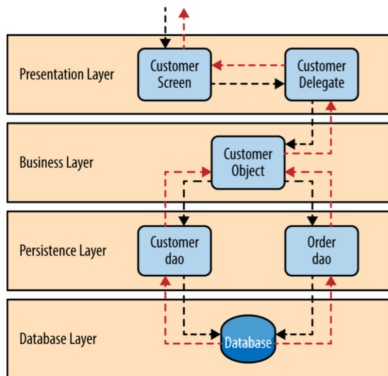


Luento 7

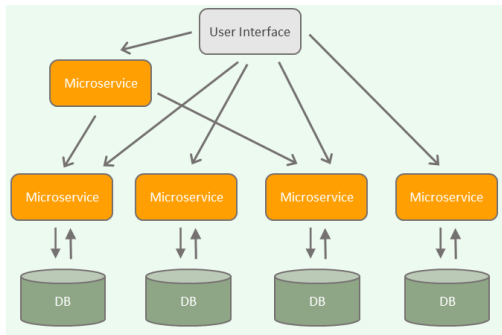
17.11.2025

Kerrosarkkitehtuuri

- *Kerros* on kokoelma toisiinsa liittyviä olioita, jotka muodostavat toiminnallisuuden suhteen loogisen kokonaisuuden



Mikropalveluarkkitehtuuri



- sovellus koostetaan useista (jopa sadoista) pienistä verkossa toimivista autonomisista palveluista